Vocheric Shrike Light FPGA + RP2040

VECTOR8_CPU Project

Complete Verilog and Python Documentation

Firmware Development Team

July 16, 2026

Contents

1 Introduction

This document provides a comprehensive line-by-line explanation of all code files in the VECTOR8_CPU project. The system consists of:

- **FPGA Design:** Custom 8-bit CPU, SPI communication, ultrasonic sensor interface, and LED driver.
- **RP2040 Software:** Python-based assembly terminal for programming the CPU.

All Verilog modules and Python scripts are documented with detailed comments explaining every line of code.

2 Section A: SPI_TARGET.V — SPI Communication Module (FPGA Side)

This module makes the FPGA act as an SPI target (slave) device that receives commands from the RP2040 microcontroller over the SPI bus.

2.1 Module Declaration & Parameters

```
module spi_target #(
  parameter CPOL   = 1'b0,
  // CPOL (Clock Polarity): When = 1'b0, SPI clock is LOW when idle.
  // When = 1'b1, clock is HIGH when idle. Here we use mode 0 (CPOL=0).

  parameter CPHA   = 1'b0,
  // CPHA (Clock Phase): When = 1'b0, data is sampled on RISING edge.
  // When = 1'b1, sampled on FALLING edge. Here mode 0 (CPHA=0).

  parameter WIDTH  = 8,
  // WIDTH: Number of bits per SPI transfer. 8 bits = 1 byte.

  parameter LSB    = 1'b0
  // LSB: When = 1'b1, Least Significant Bit is sent first.
  // When = 1'b0, Most Significant Bit is sent first. Here MSB first.

) (
  // common ports
  input                   i_clk,
  // i_clk: FPGA's main system clock (50 MHz from oscillator).

  input                   i_rst_n,
  // i_rst_n: Active-LOW reset signal. When LOW, module resets.

  // control signal
  input                   i_enable,
  // i_enable: When HIGH, SPI target is active and can receive/send data
    .

  // SPI interface ports
  input                   i_ss_n,
  // i_ss_n: SPI Slave Select (active LOW). When LOW, FPGA is selected.
```

```
32
33  input                   i_sck,
34  // i_sck: SPI Serial Clock from RP2040. Controls data timing.
35
36  input                   i_mosi,
37  // i_mosi: Master Out Slave In. Data FROM RP2040 TO FPGA.
38
39  output                  o_miso,
40  // o_miso: Master In Slave Out. Data FROM FPGA TO RP2040.
41
42  output                  o_miso_oe,
43  // o_miso_oe: Output Enable for MISO pin. HIGH when FPGA drives MISO.
44
45  // RX internal ports
46  output reg [WIDTH-1:0] o_rx_data,
47  // o_rx_data: 8-bit register holding the byte just received from
     RP2040.
48
49  output reg              o_rx_data_valid,
50  // o_rx_data_valid: Pulses HIGH for 1 clock when a full byte is
     received.
51
52  // TX internal ports
53  input      [WIDTH-1:0] i_tx_data,
54  // i_tx_data: 8-bit data that FPGA will send back to RP2040.
55
56  output                  o_tx_data_hold
57  // o_tx_data_hold: Signal telling the system to load new data into
     i_tx_data.
58 );
```

Listing 1: SPI Target Module Declaration

2.2 Internal Signal Declarations

```
1  reg               [2:0] r_ss_n_sync, r_sck_sync;
2  // r_ss_n_sync: 3-stage shift register to synchronize i_ss_n to FPGA
     clock.
3  // r_sck_sync: 3-stage shift register to synchronize i_sck to FPGA
     clock.
4  // Synchronization prevents metastability (glitches) from crossing
     clock domains.
5
6  reg [$clog2(WIDTH-1):0] r_transmision_count;
7  // Bit counter. $clog2(7) = 3 bits. Counts 0-7 for 8-bit transfers.
8
9  reg         [WIDTH-1:0] r_miso_data;
10 // Shift register holding data being transmitted out on MISO.
11
12 wire                    w_sck_r_edge, w_sck_f_edge, w_sck_edge,
     w_sck_edge_op;
13 // Edge detection wires. Will detect rising/falling edges of SPI clock
     .
```

Listing 2: Internal Signals

2.3 SPI Input Synchronization (Clock Domain Crossing)

```
always @(posedge i_clk or negedge i_rst_n) begin
  if (!i_rst_n) begin
    r_ss_n_sync <= 'b111;
    // Reset: Set all sync stages to 1 (SS inactive/high).
  end else if (i_enable) begin
    r_ss_n_sync <= {r_ss_n_sync[1:0], i_ss_n};
    // Shift in new i_ss_n value. [2:1] get old values, [0] gets new i_ss_n.
    // After 3 clocks, i_ss_n is fully synchronized to FPGA clock domain.
  end else begin
    r_ss_n_sync <= 'b111;
    // If disabled, keep SS sync register at inactive state.
  end
end

always @(posedge i_clk or negedge i_rst_n) begin
  if (!i_rst_n) begin
    r_sck_sync <= 'h0;
    // Reset: Clear SCK sync register.
  end else if (i_enable) begin
    r_sck_sync <= {r_sck_sync[1:0], i_sck};
    // Same 3-stage synchronization for SPI clock signal.
  end else begin
    r_sck_sync <= 'h0;
    // If disabled, clear SCK sync register.
  end
end
```

Listing 3: Clock Domain Crossing

2.4 Edge Detection Logic

```
assign w_sck_r_edge  = ~r_sck_sync[2] & r_sck_sync[1];
// Rising edge detection: Stage[2] was 0, Stage[1] is 1.
// This means SCK just went from LOW to HIGH.

assign w_sck_f_edge  = r_sck_sync[2] & ~r_sck_sync[1];
// Falling edge detection: Stage[2] was 1, Stage[1] is 0.
// This means SCK just went from HIGH to LOW.

assign w_sck_edge    = (CPHA^CPOL) ? w_sck_f_edge : w_sck_r_edge;
// w_sck_edge = edge where we SAMPLE incoming data (MOSI).
// For CPOL=0, CPHA=0: 0^0=0, so use RISING edge to sample.

assign w_sck_edge_op = (CPHA^CPOL) ? w_sck_r_edge : w_sck_f_edge;
// w_sck_edge_op = edge where we SHIFT/UPDATE outgoing data (MISO).
// Opposite of sample edge. For mode 0: use FALLING edge to shift out.
```

Listing 4: Edge Detection

2.5 Transmission Bit Counter

```
always @(posedge i_clk or negedge i_rst_n) begin
  if (!i_rst_n) begin
    r_transmision_count <= 'h0;
    // Reset: Clear bit counter.
  end else if (!i_enable || r_ss_n_sync[1]) begin
    r_transmision_count <= 'h0;
    // If disabled OR slave select is inactive, reset counter.
  end else if (w_sck_edge) begin
    // On every sample edge of SPI clock:
    if (r_transmision_count == WIDTH-1) begin
      r_transmision_count <= 'h0;
      // If we just received the 8th bit (count=7), wrap to 0.
    end else begin
      r_transmision_count <= r_transmision_count + 1;
      // Otherwise, increment bit counter (0->1->2->...->7).
    end
  end
end
```

Listing 5: Bit Counter

2.6 Receive Data Shift Register (MOSI → o_rx_data)

```
always @(posedge i_clk or negedge i_rst_n) begin
  if (!i_rst_n) begin
    o_rx_data <= 'h0;
    // Reset: Clear received data register.
  end else if (w_sck_edge) begin
    // On every sample edge:
    if (LSB) begin
      o_rx_data <= {i_mosi, o_rx_data[WIDTH-1:1]};
      // LSB FIRST mode: Shift RIGHT. New bit (i_mosi) goes to MSB
 position.
    end else begin
      o_rx_data <= {o_rx_data[WIDTH-2:0], i_mosi};
      // MSB FIRST mode (our case): Shift LEFT. New bit goes to LSB
 position.
    end
  end
end
```

Listing 6: Receive Shift Register

2.7 Receive Data Valid Signal

```
always @(posedge i_clk or negedge i_rst_n) begin
  if (!i_rst_n) begin
    o_rx_data_valid <= 1'b0;
    // Reset: Data valid signal is LOW.
  end else if (r_ss_n_sync[1] || (r_transmision_count == 0 &&
 w_sck_edge)) begin
    o_rx_data_valid <= 1'b0;
    // Clear valid when: SS goes inactive OR we just started a new
 byte (count=0).
  end else if (w_sck_edge && r_transmision_count == WIDTH-1) begin
```

```
      o_rx_data_valid <= 1'b1;
      // SET valid when: On sample edge AND we just received the last bit (count=7).
      // This means a complete byte has been received!
    end
  end
```

Listing 7: Data Valid Signal

2.8 Transmit Data Hold Signal

```
  assign o_tx_data_hold = (~CPHA & r_ss_n_sync[2] & ~r_ss_n_sync[1]) | (r_transmision_count == 0 & w_sck_edge_op);
  // o_tx_data_hold goes HIGH in TWO cases:
  // Case 1: (~CPHA & r_ss_n_sync[2] & ~r_ss_n_sync[1])
  //    - CPHA=0 AND SS was high (sync[2]=1) AND SS just went low (sync[1]=0)
  //    - This is the falling edge of SS (start of SPI transaction)
  //    - Load first data byte when transaction begins
  // Case 2: (r_transmision_count == 0 & w_sck_edge_op)
  //    - Bit counter is 0 AND we're on the shift/update edge
  //    - Load next data byte at the start of each new byte transfer
```

Listing 8: Transmit Hold Signal

2.9 Transmit Data Shift Register (MISO output)

```
  always @(posedge i_clk or negedge i_rst_n) begin
    if (!i_rst_n) begin
      r_miso_data <= 'h0;
      // Reset: Clear MISO shift register.
    end else if (o_tx_data_hold) begin
      r_miso_data <= i_tx_data;
      // When hold signal is HIGH: Load new data from i_tx_data into shift register.
      // This is the data that will be sent back to RP2040.
    end else if (w_sck_edge_op) begin
      // On every shift/update edge:
      if (LSB) begin
        r_miso_data <= r_miso_data >> 1;
        // LSB mode: Shift RIGHT. Bit[0] goes out on MISO, bits shift down.
      end else begin
        r_miso_data <= r_miso_data << 1;
        // MSB mode (our case): Shift LEFT. Bit[7] goes out on MISO, bits shift up.
      end
    end
  end
```

Listing 9: Transmit Shift Register

2.10 MISO Output Assignments

```
assign o_miso    = (LSB) ? r_miso_data[0] : r_miso_data[WIDTH-1];
// Output the appropriate bit on MISO:
// - LSB mode: output bit[0]
// - MSB mode: output bit[7] (our case)

assign o_miso_oe = ~r_ss_n_sync[2];
// MISO output enable: HIGH when SS is active (LOW).
// Only drive MISO when we're selected. Prevents bus contention.
```

Listing 10: MISO Output

3 Section B: TOP.V — Top-Level Module (FPGA Main Design)

This is the ROOT module that instantiates all other modules and connects them. It defines all physical pins on the FPGA.

3.1 Module Declaration with Pin Attributes

```
(* top *) module top (
  // (* top *) : Tells the synthesis tool this is the top-level module.

  (* iopad_external_pin, clkbuf_inhibit *) input clk,
  // clk: 50 MHz external crystal oscillator input.
  // clkbuf_inhibit: Don't use automatic clock buffer, use dedicated
   clock pin.

  (* iopad_external_pin *) output clk_en,
  // clk_en: Clock enable output. Always HIGH to enable clock
   distribution.

  (* iopad_external_pin *) input rst_n,
  // rst_n: Active-LOW reset button input.

  // SPI Interface pins
  (* iopad_external_pin *) input spi_ss_n,
  // spi_ss_n: SPI Slave Select from RP2040. Active LOW.

  (* iopad_external_pin *) input spi_sck,
  // spi_sck: SPI Serial Clock from RP2040.

  (* iopad_external_pin *) input spi_mosi,
  // spi_mosi: Data from RP2040 to FPGA.

  (* iopad_external_pin *) output spi_miso,
  // spi_miso: Data from FPGA to RP2040.

  (* iopad_external_pin *) output spi_miso_en,
  // spi_miso_en: Output enable for MISO pin.

  // Ultrasonic sensor pins
  (* iopad_external_pin *) input echo,
  // echo: Input from HC-SR04 ultrasonic sensor echo pin.

  (* iopad_external_pin *) output trig,
  // trig: Output to HC-SR04 trigger pin.

  (* iopad_external_pin *) output trig_en,
  // trig_en: Output enable for trigger pin.

  // Port 16 pattern LED
  (* iopad_external_pin *) output led16,
  // led16: Single LED output showing one bit of the 16-LED pattern.

  (* iopad_external_pin *) output led16_en,
  // led16_en: Output enable for LED pin.
```

```
46
47    // Direct sensor status pin
48    (* iopad_external_pin *) output object_detected_pin,
49    // object_detected_pin: Direct output showing sensor status (for
       debugging).
50
51    (* iopad_external_pin *) output object_detected_en
52    // object_detected_en: Output enable for sensor status pin.
53 );
```

Listing 11: Top Module Declaration

3.2 Output Enable Assignments

```
1   assign clk_en               = 1'b1;
2   // Always enable the clock buffer.
3
4   assign trig_en              = 1'b1;
5   // Always enable the trigger output.
6
7   assign led16_en             = 1'b1;
8   // Always enable the LED output.
9
10  assign object_detected_en   = 1'b1;
11  // Always enable the sensor status output.
```

Listing 12: Output Enables

3.3 SPI Target Instance

```
1   wire [7:0] spi_rx_data;
2   // Wire carrying the byte received from RP2040 via SPI.
3
4   wire spi_rx_valid;
5   // Wire that pulses HIGH when a complete byte is received.
6
7   reg  spi_rx_valid_d;
8   // Delayed version of spi_rx_valid. Used for edge detection.
9
10  spi_target #( .WIDTH(8) ) u_spi_target (
11    // Instantiate spi_target module with 8-bit width.
12    // u_spi_target is the instance name.
13
14    .i_clk(clk), .i_rst_n(rst_n), .i_enable(1'b1),
15    // Connect system clock, reset (always enabled).
16
17    .i_ss_n(spi_ss_n), .i_sck(spi_sck), .i_mosi(spi_mosi),
18    // Connect SPI interface pins from RP2040.
19
20    .o_miso(spi_miso), .o_miso_oe(spi_miso_en),
21    // Connect MISO output and its enable.
22
23    .o_rx_data(spi_rx_data), .o_rx_data_valid(spi_rx_valid),
24    // Connect received data and valid signal.
25
```

```
    .i_tx_data(cpu_acc), .o_tx_data_hold()
    // cpu_acc (accumulator) is sent back to RP2040.
    // o_tx_data_hold is unused (open).
  );
```

Listing 13: SPI Target Instantiation

3.4 SPI Command Decoder (2-Byte Protocol)

```
  reg [4:0] r_opcode;
  // Register storing the 5-bit opcode (first byte from RP2040).

  reg [7:0] r_data;
  // Register storing the 8-bit data value (second byte from RP2040).

  reg byte_cnt;
  // Counter: 0 = waiting for opcode, 1 = waiting for data.

  reg step_pulse;
  // Single-clock pulse that tells CPU to execute one instruction.

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      byte_cnt <= 0;
      step_pulse <= 0;
      spi_rx_valid_d <= 0;
      // Reset: Clear all control registers.
    end else begin
      spi_rx_valid_d <= spi_rx_valid;
      // Save previous valid state for edge detection.

      step_pulse <= 0;
      // Default: no step pulse this clock.

      if (spi_rx_valid && !spi_rx_valid_d) begin
        // Rising edge of spi_rx_valid: A new byte just arrived!
        if (byte_cnt == 0) begin
          r_opcode <= spi_rx_data[4:0];
          // First byte: Extract lower 5 bits as opcode.
          byte_cnt <= 1;
          // Next byte will be the data operand.
        end else begin
          r_data <= spi_rx_data;
          // Second byte: Store as data operand.
          byte_cnt <= 0;
          // Reset for next instruction pair.
          step_pulse <= 1;
          // PULSE: Tell CPU to execute this instruction!
        end
      end

      if (spi_ss_n) byte_cnt <= 0;
      // If SS goes inactive (HIGH), reset byte counter.
      // This handles aborted transfers.
    end
```

```
47  end
```

Listing 14: SPI Command Decoder

3.5 Ultrasonic Sensor Instance

```
1  wire object_detected;
2  // Wire carrying sensor detection result (1 = object detected).
3
4  ultrasonic_sensor #(.CLK_FREQ(50000000), .MAX_DIST_CM(20))
5    u_sensor (.clk(clk), .echo(echo), .trig(trig),
6              .object_detected(object_detected));
7  // Instantiate ultrasonic sensor module:
8  // - CLK_FREQ = 50 MHz (FPGA clock)
9  // - MAX_DIST_CM = 20 cm detection range
10 // - u_sensor is the instance name
11
12 assign object_detected_pin = object_detected;
13 // Drive the direct debug pin with sensor status.
```

Listing 15: Ultrasonic Sensor Instantiation

3.6 CPU Core Instance

```
1  wire [7:0] cpu_pc;
2  // Program Counter: Current instruction address (0-255).
3
4  wire [7:0] cpu_acc;
5  // Accumulator: Main 8-bit working register of the CPU.
6
7  wire [7:0] port16_pattern;
8  // 8-bit LED pattern output from CPU.
9
10 cpu_core u_cpu (
11   .clk(clk),
12   .rst_n(rst_n),
13   .i_step(step_pulse),
14   // step_pulse: Execute one instruction when this pulses HIGH.
15
16   .opcode(r_opcode),
17   // 5-bit opcode from SPI decoder.
18
19   .data_in(r_data),
20   // 8-bit data operand from SPI decoder.
21
22   .sensor_in(object_detected),
23   // Ultrasonic sensor input to CPU.
24
25   .pc(cpu_pc),
26   .acc(cpu_acc),
27   .port16_pattern(port16_pattern)
28   // Outputs: PC, accumulator value, and LED pattern.
29 );
```

Listing 16: CPU Core Instantiation

3.7 Port16 LED Pattern Sequencer

```
localparam CLK_FREQ    = 50_000_000;
// FPGA clock frequency: 50 MHz.

localparam SLOT_MS     = 250;
// Each LED position is shown for 250 milliseconds.

localparam SLOT_CYCLES = (CLK_FREQ / 1000) * SLOT_MS;
// Calculate clock cycles per slot: (50,000,000 / 1000) * 250 =
 12,500,000 cycles.

reg [31:0] slot_counter;
// Counter that counts up to SLOT_CYCLES.

reg [2:0]  bit_index;
// Index 0-7 selecting which bit of port16_pattern to display.

always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    slot_counter <= 0;
    bit_index    <= 0;
    // Reset: Clear counter and index.
  end else if (slot_counter >= SLOT_CYCLES - 1) begin
    slot_counter <= 0;
    // Counter reached limit: Reset counter.
    bit_index    <= bit_index + 1;
    // Move to next bit position (0->1->2->...->7->0).
  end else begin
    slot_counter <= slot_counter + 1;
    // Normal operation: Increment counter.
  end
end

assign led16 = port16_pattern[7 - bit_index];
// Output the selected bit. bit_index=0 shows bit[7], index=1 shows
 bit[6], etc.
// This creates a rotating display effect on the LED.
```

Listing 17: LED Pattern Sequencer

4 Section C: CPU_CORE.V — 8-Bit CPU Processor

This is the HEART of the project - a custom 8-bit CPU that executes assembly instructions received from RP2040 via SPI.

4.1 Module Declaration

```
module cpu_core (
  input  clk,
  // System clock (50 MHz).

  input  rst_n,
  // Active-LOW reset.

  input  i_step,
  // Instruction step pulse. When HIGH, execute one instruction.

  input  [4:0] opcode,
  // 5-bit operation code defining what instruction to execute.

  input  [7:0] data_in,
  // 8-bit data operand (second byte of SPI transfer).

  input        sensor_in,
  // Ultrasonic sensor input (1 = object detected).

  output reg [7:0] pc,
  // Program Counter: Points to current instruction. 0-255 range.

  output reg [7:0] acc,
  // Accumulator: Main working register for arithmetic/logic.

  output reg [7:0] port16_pattern
  // LED pattern output. Written by OUT instruction.
);
```

Listing 18: CPU Core Declaration

4.2 ALU Instance

```
  wire [7:0] alu_out;
  // ALU computation result.

  wire alu_zero;
  // HIGH when ALU result is zero.

  reg  z_flag;
  // Zero flag register. Stores result of last ALU operation.

  alu_8bit u_alu (
    .a_reg(acc),
    // First ALU operand: accumulator value.

    .data_in(data_in),
    // Second ALU operand: data from SPI.
```

```
16
17      .opcode(opcode),
18      // Operation to perform.
19
20      .out(alu_out),
21      // Computation result.
22
23      .zero(alu_zero)
24      // Zero flag output.
25  );
```

Listing 19: ALU Instantiation

4.3 CPU State Machine (Instruction Execution)

```
1   always @(posedge clk or negedge rst_n) begin
2     if (!rst_n) begin
3       pc              <= 8'h00;
4       // Reset PC to address 0.
5
6       acc             <= 8'h00;
7       // Clear accumulator.
8
9       z_flag          <= 1'b0;
10      // Clear zero flag.
11
12      port16_pattern <= 8'h00;
13      // Clear LED pattern.
14
15    end else if (i_step) begin
16      // Only execute when step_pulse is HIGH (once per instruction).
17
18      pc <= pc + 1'b1;
19      // Increment program counter to next instruction.
20
21      case (opcode)
22        // Decode opcode and execute appropriate operation.
23
24        5'h01, 5'h02, 5'h03, 5'h04, 5'h05, 5'h06,
25        5'h07, 5'h08, 5'h09, 5'h0A, 5'h0B, 5'h0C: begin
26        // Opcodes 0x01-0x0C: ALU operations (LDA, ADD, SUB, AND, OR,
    XOR, LSL, LSR, ROL, ROR, INC, DEC)
27
28          acc    <= alu_out;
29          // Store ALU result in accumulator.
30
31          z_flag <= alu_zero;
32          // Update zero flag based on result.
33        end
34
35        5'h0D: pc <= data_in;
36        // JMP (Jump): Set PC to data_in address. Unconditional branch.
37
38        5'h0E: if (z_flag) pc <= data_in;
39        // JZ (Jump if Zero): Only jump if zero flag is set.
40
41        5'h0F: if (!z_flag) pc <= data_in;
```

```
42          // JNZ (Jump if Not Zero): Only jump if zero flag is clear.
43
44          5'h10: begin
45            // SENSE: Read ultrasonic sensor.
46            acc    <= {7'b0, sensor_in};
47            // Load sensor value into accumulator (0 or 1, padded to 8
    bits).
48            z_flag <= !sensor_in;
49            // Zero flag = 1 when no object detected.
50          end
51
52          5'h12: port16_pattern <= data_in;
53          // OUT: Write data_in to LED pattern register.
54
55          default: ;
56          // Unknown opcodes: Do nothing (NOP).
57        endcase
58      end
59    end
```

Listing 20: CPU State Machine

5 Section D: ALU_8BIT.V — Arithmetic Logic Unit

The ALU performs all arithmetic and logical operations for the CPU.

5.1 Module Declaration

```
module alu_8bit (
  input  [7:0] a_reg,
  // First operand: Accumulator value.

  input  [7:0] data_in,
  // Second operand: Data from instruction.

  input  [4:0] opcode,
  // Operation selector.

  output reg [7:0] out,
  // Computation result.

  output wire zero
  // Zero flag: HIGH when out == 0.
);
```

Listing 21: ALU Declaration

5.2 ALU Operation Decoder

```
  always @(*) begin
    // Combinational logic: Output changes immediately when inputs
   change.

    case (opcode)
      5'h01: out = data_in;
      // LDA (Load): out = data_in. Load immediate value.

      5'h02: out = a_reg + data_in;
      // ADD: out = accumulator + data_in.

      5'h03: out = a_reg - data_in;
      // SUB: out = accumulator - data_in.

      5'h04: out = a_reg & data_in;
      // AND: Bitwise AND of accumulator and data_in.

      5'h05: out = a_reg | data_in;
      // OR: Bitwise OR of accumulator and data_in.

      5'h06: out = a_reg ^ data_in;
      // XOR: Bitwise XOR of accumulator and data_in.

      5'h07: out = a_reg << 1;
      // LSL (Logical Shift Left): Shift left by 1, 0 fills LSB.

      5'h08: out = a_reg >> 1;
      // LSR (Logical Shift Right): Shift right by 1, 0 fills MSB.
```

```
28
29      5'h09: out = {a_reg[6:0], a_reg[7]};
30      // ROL (Rotate Left): MSB wraps around to LSB.
31
32      5'h0A: out = {a_reg[0], a_reg[7:1]};
33      // ROR (Rotate Right): LSB wraps around to MSB.
34
35      5'h0B: out = a_reg + 1'b1;
36      // INC (Increment): Add 1 to accumulator.
37
38      5'h0C: out = a_reg - 1'b1;
39      // DEC (Decrement): Subtract 1 from accumulator.
40
41      default: out = a_reg;
42      // Default/NOP: Pass accumulator through unchanged.
43    endcase
44  end
45
46  assign zero = (out == 8'b0);
47  // Zero flag: Compare output to 0. HIGH if equal.
```

Listing 22: ALU Operation Decoder

6 Section E: ULTRASONIC_SENSOR.V — HC-SR04 Interface

This module interfaces with the HC-SR04 ultrasonic distance sensor.

6.1 Module Declaration

```
module ultrasonic_sensor #(
  parameter CLK_FREQ      = 50000000,
  // System clock frequency: 50 MHz.

  parameter TRIG_PULSE_US = 10,
  // Trigger pulse width: 10 microseconds (sensor requirement).

  parameter MAX_DIST_CM   = 20
  // Maximum detection distance: 20 centimeters.
) (
  input       clk,
  // System clock.

  input       echo,
  // Echo input from sensor (pulse width = distance).

  output reg  trig,
  // Trigger output to sensor.

  output reg  object_detected
  // Detection result: 1 = object within MAX_DIST_CM, 0 = no object.
);
```

Listing 23: Ultrasonic Sensor Declaration

6.2 Local Parameters (Computed Constants)

```
  localparam MAX_ECHO_TIME_US = (MAX_DIST_CM * 1000 * 1000) / 17150;
  // Calculate max echo time in microseconds.
  // Formula: distance = (time * speed_of_sound) / 2
  // speed_of_sound = 343 m/s = 0.0343 cm/us
  // time = (2 * distance) / 0.0343 = distance / 0.01715 us
  // For 20cm: 20 / 0.01715 = 1166 us (approx)

  localparam MAX_COUNT        = (MAX_ECHO_TIME_US * (CLK_FREQ / 1000000)
   );
  // Convert max time to clock cycles.
  // For 50MHz: 1166 * 50 = 58,300 clock cycles.

  localparam DEBOUNCE_CNT_LIMIT = 500000;
  // Debounce counter limit. At 50MHz = 10ms of stable reading required.
```

Listing 24: Local Parameters

6.3 Internal Registers

```
reg [31:0]  echo_count = 0;
// Counter measuring echo pulse width in clock cycles.

reg         echo_prev = 0;
// Previous echo state for edge detection.

reg [31:0]  trig_counter = 0;
// Counter for generating periodic trigger pulses.

reg         detect = 0;
// Raw detection result (before debouncing).

reg [31:0]  debounce_clk_cnt = 0;
// Counter for debounce timing.
```

Listing 25: Internal Registers

6.4 Trigger Pulse Generator

```
always @(posedge clk) begin
  if (trig_counter < (CLK_FREQ * 60_000 / 1_000_000))
    trig_counter <= trig_counter + 1;
  // Count up to 60ms period (sensor needs 60ms between measurements).
  // 50MHz * 60,000 / 1,000,000 = 3,000,000 cycles = 60ms.
  else
    trig_counter <= 0;
  // Reset counter after 60ms.

  if (trig_counter < (TRIG_PULSE_US * (CLK_FREQ / 1_000_000)))
    trig <= 1;
  // First 10us of each 60ms period: trig = HIGH.
  // 10 * 50 = 500 cycles = 10us.
  else
    trig <= 0;
  // Rest of the time: trig = LOW.
end
```

Listing 26: Trigger Pulse Generator

6.5 Echo Pulse Measurement

```
always @(posedge clk) begin
  echo_prev <= echo;
  // Save current echo for next edge detection.

  if (~echo_prev && echo)
    echo_count <= 0;
  // Rising edge of echo: Start counting (object detected, measuring
 distance).
  else if (echo)
    echo_count <= echo_count + 1;
  // Echo is HIGH: Count clock cycles to measure pulse width.

  if (echo_prev && ~echo) begin
    // Falling edge of echo: Measurement complete.
```

```
      if (echo_count <= MAX_COUNT)
        detect <= 1;
      // If echo was short enough, object is within range.
      else
        detect <= 0;
      // Echo was too long, object is too far or no object.
    end
  end
```

Listing 27: Echo Pulse Measurement

6.6 Debounce Logic

```
  always @(posedge clk) begin
    if ((object_detected != detect) && (debounce_clk_cnt <
   DEBOUNCE_CNT_LIMIT)) begin
      debounce_clk_cnt <= debounce_clk_cnt + 1;
      // Detection changed: Start counting debounce time.
    end else if (debounce_clk_cnt == DEBOUNCE_CNT_LIMIT) begin
      object_detected <= detect;
      // Debounce time reached: Update stable output.
      debounce_clk_cnt <= 0;
      // Reset debounce counter.
    end else begin
      debounce_clk_cnt <= 0;
      // No change: Keep counter at 0.
    end
  end
```

Listing 28: Debounce Logic

7 Section G: ASM_TERMINAL.PY — Assembly Terminal (RP2040 MicroPython)

This is the ORIGINAL Python code that runs ON THE RP2040 microcontroller. It provides an interactive terminal for entering and executing assembly code.

7.1 Imports

```
from machine import Pin, SPI
# Imports Pin and SPI classes from MicroPython's machine module.
# Pin: Controls GPIO pins (input/output, pull-ups, etc.)
# SPI: Hardware SPI bus controller for fast serial communication
# These are MicroPython-specific (not standard Python) - they directly
# control RP2040's hardware peripherals.

import time
# Standard MicroPython time module for delays (sleep_ms, sleep_us, etc.)
```

Listing 29: Imports

7.2 Pin Definitions

```
SCK, CS, MOSI, MISO, RST = 2, 1, 3, 0, 14
# Defines GPIO pin numbers on RP2040 for SPI communication with FPGA.
# These match the physical wiring on the Shrike Light board:
#   SCK  = GPIO2  -> spi_sck   (SPI Clock, output from RP2040)
#   CS   = GPIO1  -> spi_ss_n  (Chip Select, output from RP2040, active
    LOW)
#   MOSI = GPIO3  -> spi_mosi  (Master Out Slave In, output from RP2040)
#   MISO = GPIO0  -> spi_miso  (Master In Slave Out, input to RP2040)
#   RST  = GPIO14 -> rst_n     (FPGA reset, output from RP2040, active
    LOW)
```

Listing 30: Pin Definitions

7.3 Opcode Constants

```
OP_NOP = 0x00
# NOP (No Operation): Opcode 0x00. Used to read accumulator value back
    from FPGA.
# When sent, FPGA does nothing but returns current accumulator on MISO.

OP_LDA = 0x01
# LDA (Load Accumulator): Opcode 0x01. Loads immediate value into
    accumulator.

OP_ADD = 0x02
# ADD: Opcode 0x02. Adds data to accumulator.

OP_SUB = 0x03
# SUB: Opcode 0x03. Subtracts data from accumulator.

OP_AND = 0x04
```

```
# AND: Opcode 0x04. Bitwise AND with accumulator.

OP_OR  = 0x05
# OR: Opcode 0x05. Bitwise OR with accumulator.

OP_XOR = 0x06
# XOR: Opcode 0x06. Bitwise XOR with accumulator.

OP_LSL = 0x07
# LSL (Logical Shift Left): Opcode 0x07. Shifts accumulator left by 1.

OP_LSR = 0x08
# LSR (Logical Shift Right): Opcode 0x08. Shifts accumulator right by 1.

OP_INC = 0x0B
# INC (Increment): Opcode 0x0B. Adds 1 to accumulator.

OP_DEC = 0x0C
# DEC (Decrement): Opcode 0x0C. Subtracts 1 from accumulator.

OP_READ_SENSOR = 0x10
# SENSE: Opcode 0x10. Reads ultrasonic sensor into accumulator.

OP_MOV_PORT16  = 0x12
# OUT: Opcode 0x12. Writes data to LED port (port16_pattern).
```

Listing 31: Opcode Constants

7.4 Instruction Map (Dictionary)

```
INSTR = {
    "lda": (OP_LDA, True), "add": (OP_ADD, True), "sub": (OP_SUB, True),
    "and": (OP_AND, True), "or":  (OP_OR,  True), "xor": (OP_XOR, True),
    "inc": (OP_INC, False), "dec": (OP_DEC, False),
    "lsl": (OP_LSL, False), "lsr": (OP_LSR, False),
    "sense": (OP_READ_SENSOR, False),
    "out": (OP_MOV_PORT16, True),
}
# Dictionary mapping assembly mnemonics to (opcode, needs_data) tuples.
# Key:   assembly mnemonic string (lowercase)
# Value: tuple of (opcode_value, needs_data_flag)
#   needs_data=True:  instruction requires a data operand (e.g., LDA 10)
#   needs_data=False: instruction has no operand (e.g., INC, SENSE)
# Example: INSTR["lda"] returns (0x01, True)
# Example: INSTR["inc"] returns (0x0B, False)
```

Listing 32: Instruction Map

7.5 SPI Initialization

```
rst = Pin(RST, Pin.OUT, value=1)
# Creates a Pin object for GPIO14 (RST), configured as OUTPUT.
# Initial value = 1 (HIGH), meaning FPGA is NOT in reset (active-LOW).
# This pin controls the FPGA's rst_n input.

```

```
cs = Pin(CS, Pin.OUT, value=1)
# Creates a Pin object for GPIO1 (CS/SS), configured as OUTPUT.
# Initial value = 1 (HIGH), meaning FPGA is NOT selected (active-LOW).
# Must be LOW to start SPI communication with FPGA.

spi = SPI(0, baudrate=50_000, polarity=0, phase=0, bits=8,
          sck=Pin(SCK), mosi=Pin(MOSI), miso=Pin(MISO))
# Creates hardware SPI bus instance on RP2040:
#   SPI(0): Uses SPI peripheral 0 (RP2040 has 2 SPI peripherals)
#   baudrate=50_000: Clock speed = 50 kHz (slow, reliable for breadboard wiring)
#   polarity=0: CPOL=0, clock idle LOW
#   phase=0: CPHA=0, sample on rising edge
#   bits=8: 8-bit transfers
#   sck=Pin(2): Clock on GPIO2
#   mosi=Pin(3): Data out on GPIO3
#   miso=Pin(0): Data in on GPIO0
# This matches the FPGA's spi_target module configuration (Mode 0).
```

Listing 33: SPI Initialization

7.6 Function: reset_fpga()

```
def reset_fpga():
    rst.value(0); time.sleep_ms(200)
    # Pull RST pin LOW for 200ms. FPGA enters reset state.
    # All FPGA registers clear, CPU stops executing.

    rst.value(1); time.sleep_ms(300)
    # Pull RST pin HIGH. FPGA exits reset and starts operating.
    # 300ms delay ensures FPGA is fully initialized before SPI communication.
```

Listing 34: reset_fpga Function

7.7 Function: send(opcode, data)

```
def send(opcode, data):
    # Sends a 2-byte instruction to FPGA via SPI and reads back accumulator.
    # Parameters:
    #   opcode: 5-bit operation code (0x00-0x1F)
    #   data:   8-bit data operand (0x00-0xFF)
    # Returns: Current accumulator value from FPGA (read during NOP transaction)

    cs.value(0)
    # Pull Chip Select LOW. FPGA is now selected for SPI communication.
    # FPGA's spi_target sees ss_n go LOW and prepares to receive data.

    spi.write(bytes([opcode & 0x1F, data & 0xFF]))
    # Sends 2 bytes to FPGA:
    #   Byte 1: opcode & 0x1F - masks to 5 bits (matches FPGA's r_opcode[4:0])
    #   Byte 2: data & 0xFF   - ensures 8-bit value
```

```
    # spi.write() shifts out bits on MOSI while toggling SCK.
    # FPGA spi_target receives these bytes and generates step_pulse.

    cs.value(1); time.sleep_ms(20)
    # Pull Chip Select HIGH. Ends SPI transaction.
    # 20ms delay gives FPGA time to execute instruction (more than
    enough at 50MHz).
    # Also allows sensor readings to stabilize.

    rx = bytearray(2)
    # Creates a 2-byte buffer to receive data from FPGA.

    cs.value(0)
    # Pull CS LOW again for a second SPI transaction.

    spi.write_readinto(bytes([OP_NOP, 0x00]), rx)
    # Sends NOP (0x00, 0x00) and simultaneously reads 2 bytes from FPGA.
    # write_readinto() is full-duplex: sends on MOSI, receives on MISO
    at same time.
    # FPGA receives NOP (does nothing) but shifts out cpu_acc on MISO.
    # rx[0] = first byte received (garbage/ignored)
    # rx[1] = second byte received = current accumulator value from FPGA

    cs.value(1)
    # Pull CS HIGH. End of read transaction.

    return rx[1]
    # Returns the accumulator value (second byte of SPI read).
    # This is how RP2040 knows the result of each instruction!
```

Listing 35: send Function

7.8 Function: parse_val(s)

```
def parse_val(s):
    s = s.strip().lower()
    # Removes whitespace and converts to lowercase for consistent
    parsing.

    if s.startswith("0x"): return int(s, 16)
    # Hexadecimal format: "0x2A" -> 42. int() with base 16 parses hex
    string.

    if s.startswith("0b"): return int(s, 2)
    # Binary format: "0b101010" -> 42. int() with base 2 parses binary
    string.

    return int(s)
    # Decimal format: "42" -> 42. Default base 10 parsing.
```

Listing 36: parse_val Function

7.9 Function: parse_line(line)

```
def parse_line(line):
```

```
    line = line.strip()
    # Remove leading/trailing whitespace from user input.

    if not line or line.startswith(";") or line.startswith("#"):
        return None
    # Skip empty lines and comment lines (starting with ; or #).
    # Returns None to indicate "no instruction".

    parts = line.split()
    # Split line into parts by whitespace.
    # Example: "LDA 10" -> ["LDA", "10"]
    # Example: "INC" -> ["INC"]

    mnem = parts[0].lower()
    # Get mnemonic (first word) in lowercase for case-insensitive
    matching.

    if mnem == "outa":
        return ("outa", 0)
    # Special case: OUTA is not in INSTR dictionary.
    # It is handled separately in exec_line(). Returns a tuple to
    identify it.

    if mnem == "delay":
        if len(parts) < 2: return ("error", "DELAY needs value")
        # DELAY requires a time value. Error if missing.
        return ("delay", parse_val(parts[1]))
        # Parse the delay value and return ("delay", milliseconds).

    if mnem not in INSTR:
        return ("error", f"Unknown: {mnem}")
    # If mnemonic not found in instruction map, return error tuple.

    opcode, needs_data = INSTR[mnem]
    # Look up opcode and whether data operand is required.

    if needs_data:
        if len(parts) < 2: return ("error", f"{mnem.upper()} needs value
    ")
        # If instruction needs data but none provided, return error.
        return (opcode, parse_val(parts[1]))
        # Return (opcode, parsed_data_value).

    return (opcode, 0)
    # For instructions without data operand, return (opcode, 0).
    # The 0 is a placeholder; FPGA ignores it for no-data instructions.
```

Listing 37: parse_line Function

7.10 Function: exec_line(parsed, acc)

```
def exec_line(parsed, acc):
    # Executes one parsed instruction.
    # Parameters:
    #   parsed: tuple from parse_line() - (opcode_or_string, data)
    #   acc: current accumulator value (for OUTA instruction)
    # Returns: (new_acc, description_string)
```

```
    opcode, data = parsed
    # Unpack the parsed tuple.

    if opcode == "outa":
        send(OP_MOV_PORT16, acc)
        # OUTA sends the CURRENT accumulator value to LED port.
        # Uses OP_MOV_PORT16 (0x12) opcode with acc as data.
        return (acc, f"OUTA      -> LED = 0x{acc:02X}")
        # Returns same acc (unchanged) and formatted description.

    elif opcode == "delay":
        time.sleep_ms(data)
        # DELAY is handled in SOFTWARE on RP2040 (not sent to FPGA).
        # Python sleep pauses execution for specified milliseconds.
        return (acc, f"DELAY {data}ms")

    elif opcode == "error":
        return (acc, f"ERROR: {data}")
        # Pass through error message without executing.

    else:
        new_acc = send(opcode, data)
        # For all other instructions: send to FPGA via SPI.
        # send() returns the NEW accumulator value from FPGA.

        mnem = {v[0]:k for k,v in INSTR.items()}.get(opcode, "???")
        # Reverse lookup: find mnemonic string from opcode value.
        # Creates a temporary reverse dictionary: {0x01:"lda", 0x02:"add
", ...}
        # .get(opcode, "???") returns mnemonic or "???" if not found.

        if mnem in ("lda","add","sub","and","or","xor"):
            return (new_acc, f"{mnem.upper():4s} {data:3d}  -> Acc = {
new_acc}")
            # Format: "LDA   10  -> Acc = 10"

        elif mnem == "sense":
            status = "DETECTED" if new_acc else "CLEAR"
            return (new_acc, f"SENSE     -> Acc = {new_acc} ({status})")
            # Format: "SENSE     -> Acc = 1 (DETECTED)" or "Acc = 0 (
CLEAR)"

        elif mnem == "out":
            return (new_acc, f"OUT  {data:3d}  -> LED = 0x{data:02X}")
            # Format: "OUT   255  -> LED = 0xFF"

        else:
            return (new_acc, f"{mnem.upper():4s}       -> Acc = {new_acc
}")
            # Default format for INC, DEC, LSL, LSR.
```

Listing 38: exec_line Function

7.11 Function: execute_buffer(buffer, loop_mode=False)

```
def execute_buffer(buffer, loop_mode=False):
```

```
    # Executes all instructions in the buffer.
    # Parameters:
    #   buffer: list of (line_num, raw_text, parsed_tuple)
    #   loop_mode: if True, repeats indefinitely until Ctrl+C

    print("\n" + "-" * 45)
    print("  EXECUTING" + (" IN LOOP (Ctrl+C to stop)" if loop_mode else
    ""))
    print("-" * 45)
    # Print header showing execution mode.

    try:
        while True:
            acc = 0
            # Reset accumulator to 0 at start of each execution cycle.
            # (Note: FPGA accumulator also resets when buffer starts)

            for ln, raw, parsed in buffer:
                if parsed is None:
                    continue
                # Skip empty/comment lines (parsed=None).

                acc, desc = exec_line(parsed, acc)
                # Execute one instruction, get new accumulator and
   description.

                print(f"  [{ln:2d}] {desc}")
                # Print formatted output: "[ 1] LDA   10  -> Acc = 10"

            if not loop_mode:
                break
            # If not in loop mode, exit after one pass.

            print("  --- loop restart ---")
            time.sleep_ms(100)
            # In loop mode, wait 100ms and restart from first
   instruction.

    except KeyboardInterrupt:
        print("\n  [BREAK] Loop stopped by user.")
    # Catch Ctrl+C to gracefully stop loop mode.

    send(OP_MOV_PORT16, 0x00)
    # Turn off LED by sending OUT 0x00 after execution ends.

    print("-" * 45)
    print(f"  DONE. Final Acc = {acc}")
    print("-" * 45)
```

Listing 39: execute_buffer Function

7.12 Function: print_help()

```
def print_help():
    print("")
    # Prints a formatted help box showing all available instructions and
    commands.
```

```
    # Uses box-drawing characters for nice terminal display.
    # Shows: LDA, ADD, SUB, AND, OR, XOR, INC, DEC, LSL, LSR, OUT, OUTA, SENSE, DELAY
    # And terminal commands: execute, loop, clear, list, help, quit
    # (Full help text omitted for brevity; actual implementation prints detailed help.)
```

Listing 40: print_help Function

7.13 Main Program (Entry Point)

```
print("=" * 50)
print("  VECTOR8_CPU ASSEMBLY TERMINAL")
print("=" * 50)
print("  Type 'help' for commands. 'execute' to run code.")
print("=" * 50)
# Prints welcome banner when script starts.

reset_fpga()
# Reset FPGA to ensure clean state before accepting instructions.

buffer = []
# Empty list to store instructions. Each element is a tuple:
# (line_number, raw_text, parsed_tuple)

line_num = 1
# Counter for line numbers (starts at 1).

while True:
    try:
        cmd = input(f"\n[{line_num:2d}]> ").strip()
        # Prompt user with line number. Example: "[ 1]> "
        # .strip() removes leading/trailing whitespace.

    except (EOFError, KeyboardInterrupt):
        print("\nExiting...")
        send(OP_MOV_PORT16, 0x00)
        break
    # Handle Ctrl+D (EOF) or Ctrl+C by exiting cleanly.
    # Turn off LED before exiting.

    if not cmd:
        continue
    # Skip empty input.

    low = cmd.lower()
    # Convert to lowercase for command matching.

    if low in ("quit", "q", "exit"):
        send(OP_MOV_PORT16, 0x00)
        print("Goodbye!")
        break
    # QUIT command: turn off LED and exit.

    elif low == "help" or low == "h":
        print_help()
        continue
```

```
    # HELP command: show instruction reference.

    elif low == "clear":
        buffer = []
        line_num = 1
        print("  [CLEARED] Buffer empty.")
        continue
    # CLEAR command: erase all buffered instructions, reset line counter.

    elif low == "list":
        if not buffer:
            print("  [EMPTY] No code.")
        else:
            for ln, raw, _ in buffer:
                print(f"    {ln:2d}: {raw}")
        continue
    # LIST command: display all buffered instructions with line numbers.

    elif low == "execute" or low == "run" or low == "exec":
        execute_buffer(buffer, loop_mode=False)
        continue
    # EXECUTE command: run all buffered instructions ONCE.

    elif low == "loop":
        execute_buffer(buffer, loop_mode=True)
        continue
    # LOOP command: run all buffered instructions continuously.

    # If not a command, parse as assembly instruction
    parsed = parse_line(cmd)
    if parsed and parsed[0] == "error":
        print(f"  [ERROR] {parsed[1]}")
    else:
        buffer.append((line_num, cmd, parsed))
        print(f"  [OK] Line {line_num}")
        line_num += 1
    # Parse user input as assembly instruction.
    # If valid: add to buffer, print OK, increment line number.
    # If error: print error message, do not add to buffer.
```

Listing 41: Main Program

8 Appendix A: Complete Code Listings

8.1 Complete SPI_TARGET.V

```
// Full code as shown in Section A
module spi_target #(
  parameter CPOL   = 1'b0,
  parameter CPHA   = 1'b0,
  parameter WIDTH  = 8,
  parameter LSB    = 1'b0
) (
  input                   i_clk,
  input                   i_rst_n,
  input                   i_enable,
  input                   i_ss_n,
  input                   i_sck,
  input                   i_mosi,
  output                  o_miso,
  output                  o_miso_oe,
  output reg [WIDTH-1:0]  o_rx_data,
  output reg              o_rx_data_valid,
  input      [WIDTH-1:0]  i_tx_data,
  output                  o_tx_data_hold
);

  // Internal signals
  reg               [2:0] r_ss_n_sync, r_sck_sync;
  reg [$clog2(WIDTH-1):0] r_transmision_count;
  reg         [WIDTH-1:0] r_miso_data;
  wire                    w_sck_r_edge, w_sck_f_edge, w_sck_edge,
   w_sck_edge_op;

  // Synchronization
  always @(posedge i_clk or negedge i_rst_n) begin
    if (!i_rst_n) begin
      r_ss_n_sync <= 'b111;
    end else if (i_enable) begin
      r_ss_n_sync <= {r_ss_n_sync[1:0], i_ss_n};
    end else begin
      r_ss_n_sync <= 'b111;
    end
  end

  always @(posedge i_clk or negedge i_rst_n) begin
    if (!i_rst_n) begin
      r_sck_sync <= 'h0;
    end else if (i_enable) begin
      r_sck_sync <= {r_sck_sync[1:0], i_sck};
    end else begin
      r_sck_sync <= 'h0;
    end
  end

  // Edge detection
  assign w_sck_r_edge  = ~r_sck_sync[2] & r_sck_sync[1];
  assign w_sck_f_edge  = r_sck_sync[2] & ~r_sck_sync[1];
  assign w_sck_edge    = (CPHA^CPOL) ? w_sck_f_edge : w_sck_r_edge;
```

```
53  assign w_sck_edge_op = (CPHA^CPOL) ? w_sck_r_edge : w_sck_f_edge;
54
55  // Bit counter
56  always @(posedge i_clk or negedge i_rst_n) begin
57    if (!i_rst_n) begin
58      r_transmision_count <= 'h0;
59    end else if (!i_enable || r_ss_n_sync[1]) begin
60      r_transmision_count <= 'h0;
61    end else if (w_sck_edge) begin
62      if (r_transmision_count == WIDTH-1) begin
63        r_transmision_count <= 'h0;
64      end else begin
65        r_transmision_count <= r_transmision_count + 1;
66      end
67    end
68  end
69
70  // Receive shift register
71  always @(posedge i_clk or negedge i_rst_n) begin
72    if (!i_rst_n) begin
73      o_rx_data <= 'h0;
74    end else if (w_sck_edge) begin
75      if (LSB) begin
76        o_rx_data <= {i_mosi, o_rx_data[WIDTH-1:1]};
77      end else begin
78        o_rx_data <= {o_rx_data[WIDTH-2:0], i_mosi};
79      end
80    end
81  end
82
83  // Data valid signal
84  always @(posedge i_clk or negedge i_rst_n) begin
85    if (!i_rst_n) begin
86      o_rx_data_valid <= 1'b0;
87    end else if (r_ss_n_sync[1] || (r_transmision_count == 0 &&
     w_sck_edge)) begin
88      o_rx_data_valid <= 1'b0;
89    end else if (w_sck_edge && r_transmision_count == WIDTH-1) begin
90      o_rx_data_valid <= 1'b1;
91    end
92  end
93
94  // Transmit hold signal
95  assign o_tx_data_hold = (~CPHA & r_ss_n_sync[2] & ~r_ss_n_sync[1]) | (
     r_transmision_count == 0 & w_sck_edge_op);
96
97  // Transmit shift register
98  always @(posedge i_clk or negedge i_rst_n) begin
99    if (!i_rst_n) begin
100     r_miso_data <= 'h0;
101   end else if (o_tx_data_hold) begin
102     r_miso_data <= i_tx_data;
103   end else if (w_sck_edge_op) begin
104     if (LSB) begin
105       r_miso_data <= r_miso_data >> 1;
106     end else begin
107       r_miso_data <= r_miso_data << 1;
108     end
```

```
    end
  end

  // MISO output assignments
  assign o_miso    = (LSB) ? r_miso_data[0] : r_miso_data[WIDTH-1];
  assign o_miso_oe = ~r_ss_n_sync[2];

endmodule
```

Listing 42: Complete SPI Target Module

8.2 Complete TOP.V

```
(* top *) module top (
  (* iopad_external_pin, clkbuf_inhibit *) input clk,
  (* iopad_external_pin *) output clk_en,
  (* iopad_external_pin *) input rst_n,
  (* iopad_external_pin *) input spi_ss_n,
  (* iopad_external_pin *) input spi_sck,
  (* iopad_external_pin *) input spi_mosi,
  (* iopad_external_pin *) output spi_miso,
  (* iopad_external_pin *) output spi_miso_en,
  (* iopad_external_pin *) input echo,
  (* iopad_external_pin *) output trig,
  (* iopad_external_pin *) output trig_en,
  (* iopad_external_pin *) output led16,
  (* iopad_external_pin *) output led16_en,
  (* iopad_external_pin *) output object_detected_pin,
  (* iopad_external_pin *) output object_detected_en
);

  // Output enables
  assign clk_en              = 1'b1;
  assign trig_en             = 1'b1;
  assign led16_en            = 1'b1;
  assign object_detected_en  = 1'b1;

  // Internal signals
  wire [7:0] spi_rx_data;
  wire spi_rx_valid;
  reg  spi_rx_valid_d;
  reg [4:0] r_opcode;
  reg [7:0] r_data;
  reg byte_cnt;
  reg step_pulse;
  wire object_detected;
  wire [7:0] cpu_pc;
  wire [7:0] cpu_acc;
  wire [7:0] port16_pattern;

  // SPI target instantiation
  spi_target #( .WIDTH(8) ) u_spi_target (
    .i_clk(clk), .i_rst_n(rst_n), .i_enable(1'b1),
    .i_ss_n(spi_ss_n), .i_sck(spi_sck), .i_mosi(spi_mosi),
    .o_miso(spi_miso), .o_miso_oe(spi_miso_en),
    .o_rx_data(spi_rx_data), .o_rx_data_valid(spi_rx_valid),
    .i_tx_data(cpu_acc), .o_tx_data_hold()
```

```
);

// SPI command decoder
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    byte_cnt <= 0;
    step_pulse <= 0;
    spi_rx_valid_d <= 0;
  end else begin
    spi_rx_valid_d <= spi_rx_valid;
    step_pulse <= 0;
    if (spi_rx_valid && !spi_rx_valid_d) begin
      if (byte_cnt == 0) begin
        r_opcode <= spi_rx_data[4:0];
        byte_cnt <= 1;
      end else begin
        r_data <= spi_rx_data;
        byte_cnt <= 0;
        step_pulse <= 1;
      end
    end
    if (spi_ss_n) byte_cnt <= 0;
  end
end

// Ultrasonic sensor instantiation
ultrasonic_sensor #(.CLK_FREQ(50000000), .MAX_DIST_CM(20))
  u_sensor (.clk(clk), .echo(echo), .trig(trig),
            .object_detected(object_detected));
assign object_detected_pin = object_detected;

// CPU instantiation
cpu_core u_cpu (
  .clk(clk), .rst_n(rst_n), .i_step(step_pulse),
  .opcode(r_opcode), .data_in(r_data),
  .sensor_in(object_detected),
  .pc(cpu_pc), .acc(cpu_acc), .port16_pattern(port16_pattern)
);

// LED pattern sequencer
localparam CLK_FREQ    = 50_000_000;
localparam SLOT_MS     = 250;
localparam SLOT_CYCLES = (CLK_FREQ / 1000) * SLOT_MS;
reg [31:0] slot_counter;
reg [2:0]  bit_index;
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    slot_counter <= 0;
    bit_index    <= 0;
  end else if (slot_counter >= SLOT_CYCLES - 1) begin
    slot_counter <= 0;
    bit_index    <= bit_index + 1;
  end else begin
    slot_counter <= slot_counter + 1;
  end
end
assign led16 = port16_pattern[7 - bit_index];

```

```
endmodule
```

Listing 43: Complete Top Module

8.3 Complete CPU_CORE.V

```
module cpu_core (
  input  clk, rst_n, i_step,
  input  [4:0] opcode,
  input  [7:0] data_in,
  input  sensor_in,
  output reg [7:0] pc, acc, port16_pattern
);

  wire [7:0] alu_out;
  wire alu_zero;
  reg  z_flag;

  alu_8bit u_alu (
    .a_reg(acc), .data_in(data_in), .opcode(opcode),
    .out(alu_out), .zero(alu_zero)
  );

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      pc             <= 8'h00;
      acc            <= 8'h00;
      z_flag         <= 1'b0;
      port16_pattern <= 8'h00;
    end else if (i_step) begin
      pc <= pc + 1'b1;
      case (opcode)
        5'h01, 5'h02, 5'h03, 5'h04, 5'h05, 5'h06,
        5'h07, 5'h08, 5'h09, 5'h0A, 5'h0B, 5'h0C: begin
          acc    <= alu_out;
          z_flag <= alu_zero;
        end
        5'h0D: pc <= data_in;
        5'h0E: if (z_flag) pc <= data_in;
        5'h0F: if (!z_flag) pc <= data_in;
        5'h10: begin
          acc    <= {7'b0, sensor_in};
          z_flag <= !sensor_in;
        end
        5'h12: port16_pattern <= data_in;
        default: ;
      endcase
    end
  end
endmodule
```

Listing 44: Complete CPU Core

8.4 Complete ALU_8BIT.V

```
module alu_8bit (
  input  [7:0] a_reg, data_in,
  input  [4:0] opcode,
  output reg [7:0] out,
  output wire zero
);
  always @(*) begin
    case (opcode)
      5'h01: out = data_in;
      5'h02: out = a_reg + data_in;
      5'h03: out = a_reg - data_in;
      5'h04: out = a_reg & data_in;
      5'h05: out = a_reg | data_in;
      5'h06: out = a_reg ^ data_in;
      5'h07: out = a_reg << 1;
      5'h08: out = a_reg >> 1;
      5'h09: out = {a_reg[6:0], a_reg[7]};
      5'h0A: out = {a_reg[0], a_reg[7:1]};
      5'h0B: out = a_reg + 1'b1;
      5'h0C: out = a_reg - 1'b1;
      default: out = a_reg;
    endcase
  end
  assign zero = (out == 8'b0);
endmodule
```

Listing 45: Complete ALU

8.5 Complete ULTRASONIC_SENSOR.V

```
module ultrasonic_sensor #(
  parameter CLK_FREQ      = 50000000,
  parameter TRIG_PULSE_US = 10,
  parameter MAX_DIST_CM   = 20
) (
  input       clk, echo,
  output reg  trig, object_detected
);

  localparam MAX_ECHO_TIME_US = (MAX_DIST_CM * 1000 * 1000) / 17150;
  localparam MAX_COUNT        = (MAX_ECHO_TIME_US * (CLK_FREQ / 1000000)
   );
  localparam DEBOUNCE_CNT_LIMIT = 500000;

  reg [31:0]  echo_count = 0;
  reg         echo_prev = 0;
  reg [31:0]  trig_counter = 0;
  reg         detect = 0;
  reg [31:0]  debounce_clk_cnt = 0;

  always @(posedge clk) begin
    if (trig_counter < (CLK_FREQ * 60_000 / 1_000_000))
      trig_counter <= trig_counter + 1;
    else
      trig_counter <= 0;
    if (trig_counter < (TRIG_PULSE_US * (CLK_FREQ / 1_000_000)))
      trig <= 1;
```

```
    else
      trig <= 0;
  end

  always @(posedge clk) begin
    echo_prev <= echo;
    if (~echo_prev && echo)
      echo_count <= 0;
    else if (echo)
      echo_count <= echo_count + 1;
    if (echo_prev && ~echo) begin
      if (echo_count <= MAX_COUNT)
        detect <= 1;
      else
        detect <= 0;
    end
  end

  always @(posedge clk) begin
    if ((object_detected != detect) && (debounce_clk_cnt <
   DEBOUNCE_CNT_LIMIT)) begin
      debounce_clk_cnt <= debounce_clk_cnt + 1;
    end else if (debounce_clk_cnt == DEBOUNCE_CNT_LIMIT) begin
      object_detected <= detect;
      debounce_clk_cnt <= 0;
    end else begin
      debounce_clk_cnt <= 0;
    end
  end
endmodule
```

Listing 46: Complete Ultrasonic Sensor

8.6 Complete ASM_TERMINAL.PY

```
from machine import Pin, SPI
import time

SCK, CS, MOSI, MISO, RST = 2, 1, 3, 0, 14

OP_NOP = 0x00
OP_LDA = 0x01
OP_ADD = 0x02
OP_SUB = 0x03
OP_AND = 0x04
OP_OR  = 0x05
OP_XOR = 0x06
OP_LSL = 0x07
OP_LSR = 0x08
OP_INC = 0x0B
OP_DEC = 0x0C
OP_READ_SENSOR = 0x10
OP_MOV_PORT16  = 0x12

INSTR = {
    "lda": (OP_LDA, True), "add": (OP_ADD, True), "sub": (OP_SUB, True),
    "and": (OP_AND, True), "or":  (OP_OR,  True), "xor": (OP_XOR, True),
```

```
    "inc": (OP_INC, False), "dec": (OP_DEC, False),
    "lsl": (OP_LSL, False), "lsr": (OP_LSR, False),
    "sense": (OP_READ_SENSOR, False),
    "out": (OP_MOV_PORT16, True),
}

rst = Pin(RST, Pin.OUT, value=1)
cs = Pin(CS, Pin.OUT, value=1)
spi = SPI(0, baudrate=50_000, polarity=0, phase=0, bits=8,
          sck=Pin(SCK), mosi=Pin(MOSI), miso=Pin(MISO))

def reset_fpga():
    rst.value(0); time.sleep_ms(200)
    rst.value(1); time.sleep_ms(300)

def send(opcode, data):
    cs.value(0)
    spi.write(bytes([opcode & 0x1F, data & 0xFF]))
    cs.value(1); time.sleep_ms(20)
    rx = bytearray(2)
    cs.value(0)
    spi.write_readinto(bytes([OP_NOP, 0x00]), rx)
    cs.value(1)
    return rx[1]

def parse_val(s):
    s = s.strip().lower()
    if s.startswith("0x"): return int(s, 16)
    if s.startswith("0b"): return int(s, 2)
    return int(s)

def parse_line(line):
    line = line.strip()
    if not line or line.startswith(";") or line.startswith("#"):
        return None
    parts = line.split()
    mnem = parts[0].lower()
    if mnem == "outa":
        return ("outa", 0)
    if mnem == "delay":
        if len(parts) < 2: return ("error", "DELAY needs value")
        return ("delay", parse_val(parts[1]))
    if mnem not in INSTR:
        return ("error", f"Unknown: {mnem}")
    opcode, needs_data = INSTR[mnem]
    if needs_data:
        if len(parts) < 2: return ("error", f"{mnem.upper()} needs value
")
        return (opcode, parse_val(parts[1]))
    return (opcode, 0)

def exec_line(parsed, acc):
    opcode, data = parsed
    if opcode == "outa":
        send(OP_MOV_PORT16, acc)
        return (acc, f"OUTA      -> LED = 0x{acc:02X}")
    elif opcode == "delay":
        time.sleep_ms(data)
```

```
        return (acc, f"DELAY {data}ms")
    elif opcode == "error":
        return (acc, f"ERROR: {data}")
    else:
        new_acc = send(opcode, data)
        mnem = {v[0]:k for k,v in INSTR.items()}.get(opcode, "???")
        if mnem in ("lda","add","sub","and","or","xor"):
            return (new_acc, f"{mnem.upper():4s} {data:3d}  -> Acc = {new_acc}")
        elif mnem == "sense":
            status = "DETECTED" if new_acc else "CLEAR"
            return (new_acc, f"SENSE     -> Acc = {new_acc} ({status})")
        elif mnem == "out":
            return (new_acc, f"OUT  {data:3d}  -> LED = 0x{data:02X}")
        else:
            return (new_acc, f"{mnem.upper():4s}       -> Acc = {new_acc}")

def execute_buffer(buffer, loop_mode=False):
    print("\n" + "-" * 45)
    print("  EXECUTING" + (" IN LOOP (Ctrl+C to stop)" if loop_mode else ""))
    print("-" * 45)
    try:
        while True:
            acc = 0
            for ln, raw, parsed in buffer:
                if parsed is None:
                    continue
                acc, desc = exec_line(parsed, acc)
                print(f"  [{ln:2d}] {desc}")
            if not loop_mode:
                break
            print("  --- loop restart ---")
            time.sleep_ms(100)
    except KeyboardInterrupt:
        print("\n  [BREAK] Loop stopped by user.")
    send(OP_MOV_PORT16, 0x00)
    print("-" * 45)
    print(f"  DONE. Final Acc = {acc}")
    print("-" * 45)

def print_help():
    print("")
    print("  Available instructions:")
    print("    LDA  <val>   ADD  <val>   SUB  <val>")
    print("    AND  <val>   OR   <val>   XOR  <val>")
    print("    INC          DEC          LSL          LSR")
    print("    OUT  <val>   OUTA         SENSE")
    print("    DELAY <ms>")
    print("")
    print("  Terminal commands:")
    print("    execute   - run code once")
    print("    loop      - run code continuously")
    print("    clear     - clear buffer")
    print("    list      - show buffered instructions")
    print("    help      - this help")
    print("    quit      - exit")
```

```
135
136 print("=" * 50)
137 print("  VECTOR8_CPU ASSEMBLY TERMINAL")
138 print("=" * 50)
139 print("  Type 'help' for commands. 'execute' to run code.")
140 print("=" * 50)
141
142 reset_fpga()
143 buffer = []
144 line_num = 1
145
146 while True:
147     try:
148         cmd = input(f"\n[{line_num:2d}]> ").strip()
149     except (EOFError, KeyboardInterrupt):
150         print("\nExiting...")
151         send(OP_MOV_PORT16, 0x00)
152         break
153
154     if not cmd:
155         continue
156
157     low = cmd.lower()
158     if low in ("quit", "q", "exit"):
159         send(OP_MOV_PORT16, 0x00)
160         print("Goodbye!")
161         break
162     elif low == "help" or low == "h":
163         print_help()
164         continue
165     elif low == "clear":
166         buffer = []
167         line_num = 1
168         print("  [CLEARED] Buffer empty.")
169         continue
170     elif low == "list":
171         if not buffer:
172             print("  [EMPTY] No code.")
173         else:
174             for ln, raw, _ in buffer:
175                 print(f"    {ln:2d}: {raw}")
176         continue
177     elif low == "execute" or low == "run" or low == "exec":
178         execute_buffer(buffer, loop_mode=False)
179         continue
180     elif low == "loop":
181         execute_buffer(buffer, loop_mode=True)
182         continue
183
184     parsed = parse_line(cmd)
185     if parsed and parsed[0] == "error":
186         print(f"  [ERROR] {parsed[1]}")
187     else:
188         buffer.append((line_num, cmd, parsed))
189         print(f"  [OK] Line {line_num}")
190         line_num += 1
```

Listing 47: Complete Assembly Terminal

9 Appendix B: Instruction Set Reference

The following table summarizes the VECTOR8_CPU instruction set, derived from the opcode constants defined in the RP2040 assembly terminal.

Mnemonic	Opcode (hex)	Operand	Description
LDA	0x01	8-bit value	Load immediate value into accumulator
ADD	0x02	8-bit value	Add value to accumulator
SUB	0x03	8-bit value	Subtract value from accumulator
AND	0x04	8-bit value	Bitwise AND accumulator with value
OR	0x05	8-bit value	Bitwise OR accumulator with value
XOR	0x06	8-bit value	Bitwise XOR accumulator with value
LSL	0x07	None	Logical shift left accumulator
LSR	0x08	None	Logical shift right accumulator
INC	0x0B	None	Increment accumulator by 1
DEC	0x0C	None	Decrement accumulator by 1
SENSE	0x10	None	Read ultrasonic sensor into accumulator
OUT	0x12	8-bit value	Write value to LED port

Table 1: VECTOR8_CPU Instruction Set

Note: The instructions JMP, JZ, JNZ are also implemented in the CPU but are not exposed via the assembly terminal's mnemonic map. They can be used by sending raw opcodes (0x0D, 0x0E, 0x0F) if needed.

10 Appendix C: Hardware Pin Mapping

The table below shows the physical connections between the RP2040 and the FPGA.

RP2040 GPIO	Signal	Description
GPIO2	SCK	SPI Clock (output)
GPIO1	CS	SPI Chip Select (active low output)
GPIO3	MOSI	SPI Data Out (output)
GPIO0	MISO	SPI Data In (input)
GPIO14	RST	FPGA Reset (active low output)

Table 2: RP2040 to FPGA SPI Connections

11 Appendix D: Usage Instructions

11.1 Flashing the FPGA

1. Compile the Verilog design to a bitstream file (e.g., `FPGA_bitstream_MCU.bin`).
2. Use the `main.py` script (or the `shrike` library) to flash the FPGA.
3. After flashing, the FPGA will reconfigure with the VECTOR8_CPU design.

11.2 Running the Assembly Terminal

1. Upload `asm_terminal.py` to the RP2040 (e.g., using Thonny).
2. Connect the RP2040 via USB and run the script.
3. Type assembly instructions line by line.
4. Use `execute` to run the buffered code once, or `loop` to run it continuously.